

Unit-3

Classical Analysis

Chamundeswari Arumugam
Professor, Dept of CSE
SSN College of Engineering, Chennai

February 2020

Outline-Classical Analysis

- Structured analysis
- Flow oriented modeling
 - Data flow modeling
 - Control flow modeling
- Behavioral model
- Petrinets

Structured Analysis

Why do we need requirement model or structured analysis ?

- First technical representation of the system
- Models help Us to see things that might be missing.
- Depict system specification with a combination of diagrams and structure
- Models create a visual representation of whats expected.
- Requirements models allow us to organize our data in multiple ways.

Models developed in structured analysis

- Data model eg. ER diagram, DFD.
- Function model eg. Use case diagram
- Behavior model eg. STD

Data dictionary

- A repository that contains descriptions of all data objects consumed or produced by the software.

Requirement model

- **ER diagram** - depicts relationship between data objects. Data object description contain the description of each data.
- **DFD** - represents how data are transformed as they move through the system. Depicts the functions(and sub functions) that transform the data flow. Process specification contain the description of each function.
- **STD** - basis for system behavior model. Represents the behavior (called state) of the system and transistions(actions) made from one state to another state. Control specification description about the control aspects of the software.

Fig : structure of the analysis model



Flow oriented modeling

Creating a Data Flow Model

- DFD is widely used in requirement analysis.
- It takes input, process output view of the system.
- Data objects flow into the software are transferred by processing elements and resultant data objects flow out of the software.
- Notation : Arrow represent data object, circle represent transformation.
- Level 0 DFD or context diagram - system as a whole.
- Level 1 and level 2 DFD is the refinement of the system.

Guidelines of DFD

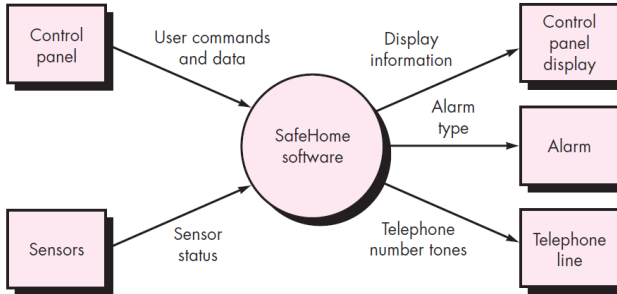
- Level 0 data flow diagram depicts software system
- Processing input and output should be carefully noted.
- Refinement should begin at next level.
- Arrows and bubbles should be labeled with meaningful names.
- Information flow continuity must be maintained from level to level.

Flow oriented modeling

Sample Data Flow Diagram

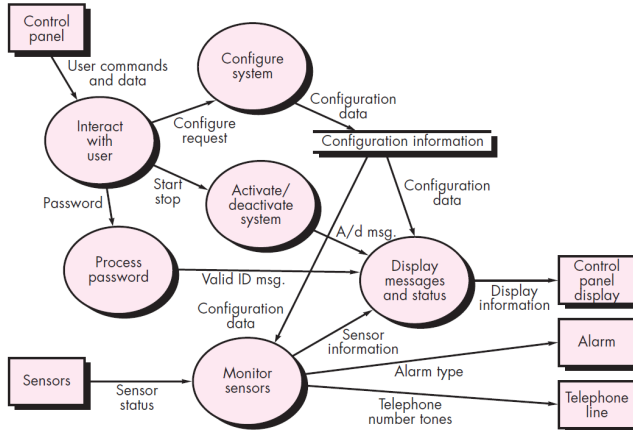
- Sample DFD diagram for SafeHome software.

Fig : Level 0 DFD /Context diagram



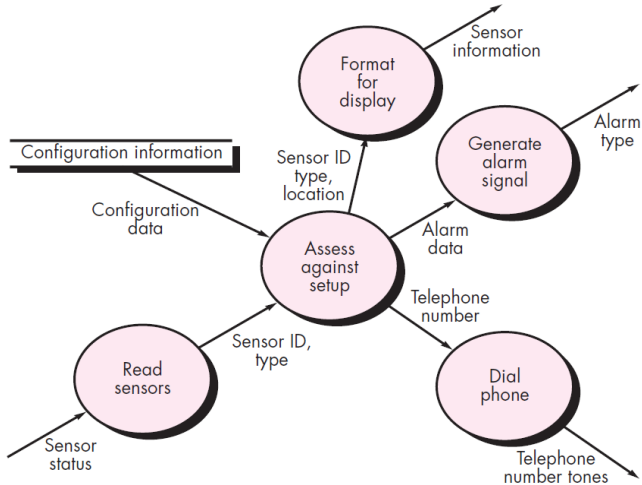
Flow oriented modeling

Fig:Level 1 DFD-SafeHome security function



Flow oriented modeling

Fig:Level 2 DFD-SafeHome security function



Flow oriented modeling

Case Study-1: Web-based pothole tracking and repair system (PHTRS)

Citizens can log onto a website and report the location and severity of potholes. As potholes are reported they are logged within a public works department repair system and are assigned an identifying number, stored by street address, size (on a scale of 1 to 10), location (middle, curb, etc.), district (determined from street address), and repair priority (determined from the size of the pothole). Work order data are associated with each pothole and include pothole location and size, repair crew identifying number, number of people on crew, equipment assigned, hours applied to repair, hole status (work in progress, repaired, temporary repair, not repaired), amount of filler material used, and cost of repair (computed from hours applied, number of people, material and equipment used). Finally, a damage file is created to hold information about reported damage due to the pothole and includes citizens name, address, phone number, type of damage, and dollar amount of damage. PHTRS is an online system; all queries are to be made interactively.

Case study : Web-based pothole tracking and repair system (PHTRS)

- Draw the ER diagram
- Draw the DFD level 0, 1 and 2.

Flow oriented modeling

Case Study

1. Network-based course registration system for your university.
2. A Web-based order-processing system for a computer store.
3. A simple invoicing system for a small business.
4. An Internet-based cookbook that is built into an electric range or microwave.

Develop the requirement model

Define the problem statement that states the user and system requirements for anyone of the activity. Develop an entity-relationship diagram that describes data objects, relationships, and attributes.

Flow oriented modeling

Creating a Control Flow Model

- Control flow can be used by applications that are driven by events rather than data and process information with heavy concern for time and performance.
- Such applications require the use of control flow modeling in addition to data flow modeling.

CFM-Control Specification

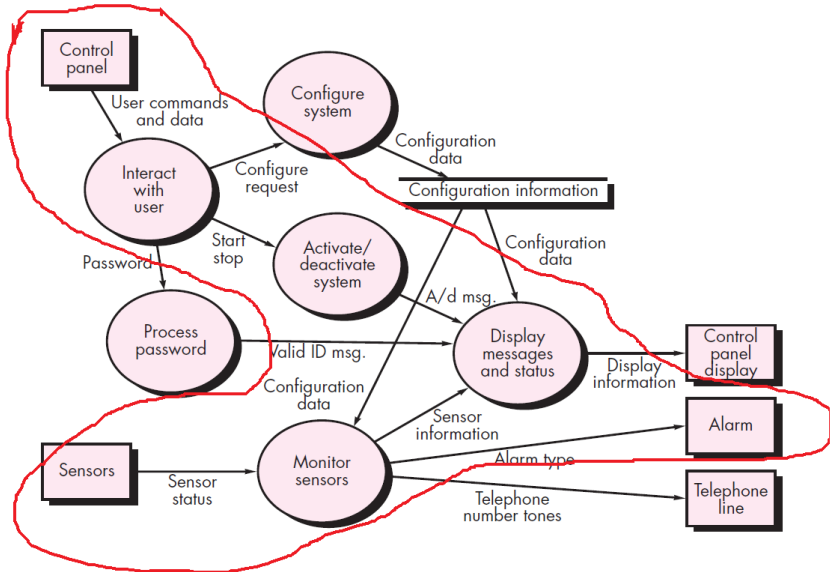
- A control specification (CSPEC) represents the behavior of the system. A state diagram indicates how the system responds to events.
- Fig A represents the state diagram and the transitions from one state to another.Eg.Operating system
- CSPEC doesn't provide information about the inner working of the processes that are activated as a result of the behavior.

CFM-Control Specification

- The process specification (PSPEC) is used to describe all flow model processes that appear at the final level of refinement
- The content of the process specification can include narrative text, a program design language (PDL) description of the process algorithm, mathematical equations, tables, or UML activity diagrams. Fig B represents the activity diagram.

Flow oriented modeling

Fig:Level 2 DFD-SafeHome security function

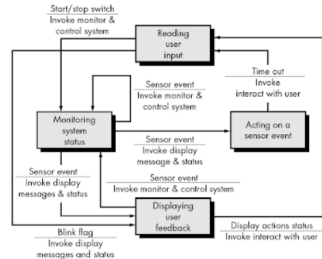


Flow oriented modeling

Process Activation Table

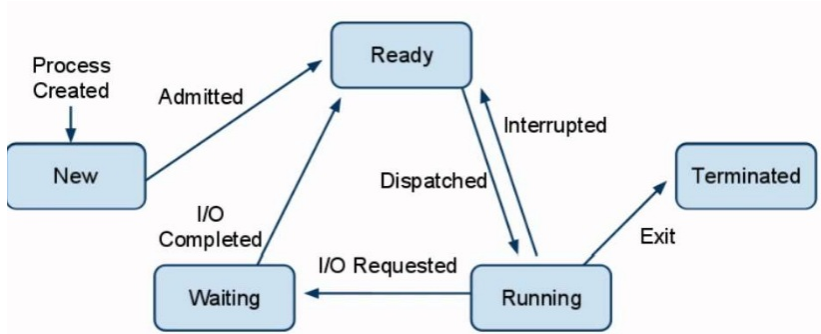
input events						
sensor event	0	0	0	0	1	0
blink flag	0	0	1	1	0	0
start stop switch	0	1	0	0	0	0
display action status complete	0	0	0	1	0	0
in-progress	0	0	1	0	0	0
time out	0	0	0	0	0	1
output						
alarm signal	0	0	0	0	1	0
process activation						
monitor and control system	0	1	0	0	1	1
activate/deactivate system	0	1	0	0	0	0
display messages and status	1	0	1	1	1	1
interact with user	1	0	0	1	0	1

State diagram



Flow oriented modeling

Fig A : OS : state diagram



Behavior Modeling

Behavior modeling

- Transition to dynamic behavior of the system or product
- Behavior of the system as a function of specific events and time.
- It indicates how software will respond to external events or stimuli.

Steps to create dynamic behavior model

- Evaluate all use cases to fully understand the sequence of interaction within the system.
- Identify events that drive the interaction sequence and understand how these events relate to specific objects.
- Create a sequence for each use case.
- Build a state diagram for the system.
- Review the behavioral model to verify accuracy and consistency.

Identifying Events with the Use Case

- Two different characterizations of states must be considered:
(1) the state of each class as the system performs its function
(2) the state of the system as observed from the outside as the system performs its function.

Fig : State diagram

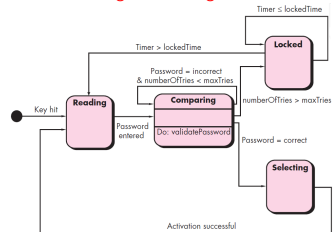
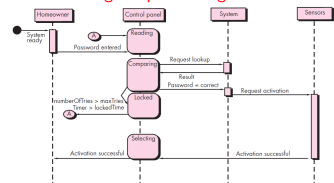


Fig: Sequence diagram



- [1] RogerS. Pressman.
"Software Engineering a Practitiner's Approach"".
Seventh Edition, McGraw Hill Higher Education, 2010.